1 MAN COMPANY

Agentic Engineering

10 DOs & Don'ts

如何用 AI Agent 跑出整个公司的产能

Startup4Chinese
Meetup 2026

10	实战要点 Key Practices
2	主力工具 Core AI Tools
1	人 = 整个团队 1 Person = Full Team

github.com/bryanzk/MyCodexEnv

你一个人，
但需要做
5 个人的事。

这不是抱怨，这是 1 Man Company 的现实。

那么，这件事真的可以做到吗？

一个人要扮演的角色

产品 / PM	需求分析 · 优先级
BA	业务建模 · 验收标准
工程师 / Dev	架构 · 编码 · 测试
QA	质量把控 · 回归验证
DevOps	部署 · 环境 · 上线

Harvard Business Review · 2026 | 200 名工程师 · 历时 9 个月

用了 AI，
工作反而更多了。

01

速度加快，任务随之增加

管理层看到效率提升，立即分配更多工作

02

工作时间自发延长

不是被要求的——是自己觉得「还能再做一点」

03

多线程切换，从未真正完成

同时管理 AI 多个线程，注意力永远碎片化

问题不在 AI，
在于你怎么用它。

大多数人把 AI 当助手——结果 AI 放大了原有的工作量

Agentic Engineering 是把 AI 当团队——重新定义你和工作的关系

不是做得更多，是选择做对的事

诺伯特·维纳 · 1948 · Cybernetics · from κυβερνήτης — 舵手

三次，同一个模式。

每一次，都是因为有人在新的层面闭合了反馈回路。

1780s

瓦特调速器

Watt's Governor

以前	工人手动拧阀门
现在	**工人设计调速器**

2010s

Kubernetes

Declarative Infra

以前	工程师手动重启服务
现在	**工程师写规格说明**

2020s

AI Agent

Harness Engineering

以前	工程师手写代码
现在	**工程师设计 Harness**

你不再亲手拧阀门，而是掌舵。 不是工作消失了——是工作的层次升高

“

任何能方法论化的知识都会被 AI 取代。

Any knowledge that can be methodologized will be replaced by AI.

The 1 Man Company

一个人，跑出整个工程团队的产能 · One person. Full team output.

C

Claude

Product / BA

拆解业务需求
澄清边界与前提
输出 MVP 方案

Requirements · Scope · MVP

C

Codex

Engineering Team

规划任务
测试驱动开发
闭环到 Git

Plan · TDD · Ship

S

Skill

Engineering Culture

最佳实践资产化
跨项目复用
团队标准固化

Reuse · Scale · Standards

G

GitHub

DevOps / Sync

多机环境同步
版本控制
持续交付

Sync · Version · Deploy

为什么「现在」是关键节点

代码库不是没有反馈回路——只是从未在最重要的那层闭合过。直到现在。

语法层	**编译器** Compiler	能编译吗?	70+ 年
行为层	**测试套件** Test Suite	能通过测试吗?	30+ 年
风格层	**Linter** Linter	符合规范吗?	40+ 年
判断层	**LLM** LLM / Agent	这个方向对吗?	🚀 现在

你的新工作：在判断层定义「正确」 — 架构文档、自定义 Lint 规则、黄金范例

Framework: Claude → Codex → Ship

以「可验证 · 可复现 · 可上线」为核心标准的完整工程链路 · End-to-end pipeline: Define → Build → Deliver

01 Claude

What

问题定义

- **约束三要素** 3-element prompt
- **MVP 输出** MVP output
- **业务需求拆解** Requirement decomposition
- **边界与决策澄清** Clarify boundaries

02 Codex 5.3

How

工程落地

- **Plan Mode** Plan before code
- **测试先行 (TDD)** Test-Driven Development
- **根因分析** Root cause analysis
- **版本闭环 Git** Git-closed loop

03 Review + Git

Ship

交付验收

- **代码审查 Skill** Code review skill
- **验收测试** Acceptance testing
- **文档同步** Doc sync
- **合并上线** Merge & deploy

语言是思维的边界。

The limits of my language
mean the limits of my world.

— 路德维希·维特根斯坦 / *Ludwig Wittgenstein, Tractatus Logico-Philosophicus, 1921*

第 1 章 · Chapter 1

重新学会提问

Rethink How You Ask

大多数人用 AI 的第一个错误，不是选错工具，是提错问题。 学会「三要素提问」，才能让 AI 真正为你所用。

01

提示词三要素：目标 + 上下文 + 最佳实践

The 3-Element Prompt: Goal · Context · Best Practices

DO

- 始终提供三要素：目标（Why）/ 上下文（Context）/ 约束（How）
- 强制引用最佳行业实践作为约束基准
- 明确说明你的业务场景和系统边界
- 效果：第一次就收敛到专属你业务的方案

DON'T

你有没有问过 AI「这个该怎么做」，然后收到一个和你业务完全无关的通用答案？

- 无上下文 = AI 只能给通用框架
- AI 无法知道你的系统架构、业务约束
- 结果：多轮反复浪费 token 也浪费时间

02

元提示词：提问前先优化提示词

Meta-Prompt: Refine Your Question Before Asking It

DO

- 用「元提示词」对初版提示词做二次优化再正式提问
- 四维度：清晰度 / 专业度 / 结构化 / 模型适应性
- 让 AI 先「审视问题」再「回答问题」
- 效果：从 ChatGPT 输出 24 条 → Claude 聚焦为 8 条

DON'T

你有没有收到过 *AI* 给的 *24* 条建议，每条都「差不多能用」，但没有一条真的对你有效?

- 未优化的提示词产出宽泛发散的结果
- 24 条验收标准：看起来全，实际难落地
- 无法针对你的具体业务场景

03

MVP 优先：让 AI 聚焦，不要让 AI 铺开

MVP First: Focus the AI, Avoid Over-Generation

DO

- 需求细节不明确时，明确要求输出 MVP 版本
- 关键词：「只给最小可验证版本，聚焦 3 个核心约束」
- 让复杂讨论快速收敛到可直接执行的任务包
- 效果：避免「看起来很全、跑起来不行」的文档陷阱

DON'T

你有没有要求 *AI* 给方案，收到一份面面俱到的文档，然后不知道从哪里开始执行?

- 过度完整的输出 = 庞大难以理解
- 24 条难以落地的验收标准
- 验证成本高，工程落地反而更慢

第 2 章 · Chapter 2

让工程真正落地

Make Engineering Real

工具选对了，工程落地靠的是工作方式。 Claude 定义问题，Codex 计划并执行，Skill 保证质量。

04

用 Claude 做业务需求拆解

Use Claude to Decompose Business Requirements

DO

- 让 Claude 主动找出你未言明的前提假设
- 工作模式：找矛盾 → 呈现 → 确认 → 才给结论
- 逼出你没意识到的隐含决策点
- 效果：每条结论都有对话节点溯源，不是通用模板

DON'T

你有没有让 *Codex* 写了三小时代码，最后发现方向根本不对，只能推倒重来?

- 跳过「澄清边界」，实现层返工率极高
- 隐含决策（如「近似 vs 精确套利」）会在代码里爆炸
- 通用回答 ≠ 你业务的答案

05

工程任务先用 Plan Mode，再执行

Plan Mode First — Then Execute

DO

- 对有复杂度的工程任务，先用 Codex Plan Mode 规划
- 锁定计划 → 确认 → 推进，少来回确认
- Codex 5.3 Plan Mode 更擅长跨 3 层以上寻找根因
- 效果：复杂多步骤任务按预期落地，不跑偏

DON'T

你有没有和 AI 边聊边写，写到一半突然发现逻辑早就漂走了？

- 多文件多步骤任务容易方向漂移
- 没有计划的执行 = 高速开车不看路
- 执行到一半发现方向错了 → 推倒重来

06

使用社区 Skill 自动调用最佳实践

Load Community Skills for Automated Best Practices

DO

- 加载社区 Skill（如 github.com/obra/superpowers）
- 涵盖：TDD / 系统调试 / 验收验证 / 代码审查
- Codex 根据问题自动调用对应 Skill
- 效果：工程最佳实践开箱即用，不需要每次手动说明

DON'T

> 你有没有对同一个 *AI* 解释了五次「我们要用 *TDD*」，但每次新对话它都好像忘了？

- 重复性工程动作消耗大量上下文 token
- “你已经教过我了，但我不记得”
- 无法积累工程护城河，每个项目归零重启

07

提炼你自己的专属 Skill

Build Your Own Reusable Custom Skills

DO

- 把反复有效的提示词模式固化为专属 Skill
- 例：Repo Branch Governance Skill（分支治理全自动化）
- 个人经验 → 跨项目可复用资产
- 效果：工程护城河持续积累，团队标准可复制

DON'T

你有没有想起「上个项目有个提示词特别好用」，但翻遍聊天记录，再也找不到？

- 下一个项目从零开始，重复踩同样的坑
- “我记得上次用过这个方法……但忘了”
- 个人经验无法沉淀，团队无法复制

“

让你的判断力变得机器可读。

Make your judgment machine-readable.

— *Harness 工程 / Harness Engineering — OpenAI, 2025*

第 3 章 · Chapter 3

把经验变成资产

Turn Experience into Assets

1 Man Company 的护城河不在工具，在于你积累的 Skill 体系。 让每一次实践都沉淀为可复用的能力。

08

GitHub 多机同步 Skill 运行环境

Sync Your Skill Environment via GitHub

DO

- 把 Skill 环境上传 GitHub，多机即插即用
- 参考：github.com/bryanzk/MyCodexEnv
- 换机器、新环境 → 一行命令恢复全套配置
- 效果：1 Man Company 无单点故障

DON'T

你有没有换过一台电脑，然后花了大半天重新配环境，最后还没配全？

- 换机器 = 从头配置，这是单点故障
- “只有我那台电脑能跑这套环境”
- 单点失效 = 单人公司的致命弱点

09

用可视化输出 Skill 提升协作效率

Visual Output Skills Improve Collaboration

DO

- 使用 Visual Explainer Skill 输出表格、对比图、状态矩阵
- 例：「当前代码，验证哪些已实现，哪些还没有，给我表格」
- 给出更多上下文，让协作者快速抓住决策点
- 效果：减少沟通成本，加速跨角色决策

DON'T

你有没有给协作者发过一段 2000 字的分析，然后对方说「能不能给我一张表？」

- 文字缺乏结构，协作者需要再加工
- “你看完这 2000 字，你记住了什么？”
- 高密度文字 = 低效沟通，尤其多人协作时

让 AI 自动完成跨工具 Skill 转换

Auto-Convert Skills Across AI Tools

DO

- superpowers 本为 Claude Code 设计，可让 Codex 自动转换
- 直接复用社区资产，零手动迁移
- “让 AI 帮你迁移 AI 工具的配置” = 元层面的 Agentic
- 效果：工具升级或切换时，Skill 资产无缝延续

DON'T

你有没有从一个 AI 工具切换到另一个，发现之前所有积累全部要手动重做?

- 这正是 Agentic Engineering 应该消灭的重复劳动
- 如果你在手动做这件事，说明你还没用好 AI
- 工具迁移本身也可以 Agentic

The 1 Man Company Stack

选对工具，用对场景，一个人跑出团队产能 · Right tool, right context, full team output.

工具 / Tool	角色 / Role	使用场景 / When to Use
Claude	Product / BA	模糊需求 → 结构化方案 · Vague idea → Structured spec
Codex 5.3	Engineering Team	方案确认 → 代码交付 · Plan → Build → Test → Merge
Raycast AI	Quick Helper	快速查询 / 文本处理 · Quick lookup, no context switch
GitHub	DevOps / Sync	Skill 环境多机同步 · Sync environment across machines
Skill 体系	Engineering Culture	最佳实践自动化 · Personal practices → Team-scale assets

3 Core Principles · 三大交付标准

Agentic Engineering 的最终交付标准 · The non-negotiable standards for every delivery

验

可验证

Verifiable

每个输出都有对应的测试或验收标准
结果可追溯，不接受「看起来没问题」

Every output has a test or acceptance criterion. Results are traceable.

复

可复现

Reproducible

任何人在任何机器上运行同样的流程
都能得到同样的结果，环境即代码

Same process, same machine or not, same result. Environment as code.

线

可上线

Shippable

交付物达到生产就绪标准
能跑、能测、能合并、能回滚

Production-ready: runnable, testable, mergeable, rollback-safe.

Agentic Engineering

不是让你工作更多，

是让你选择做对的事。

你一个人，现在你知道怎么做了。

延伸思考

现实世界能被完全总结为方法论吗?

- 如果不能，不能的部分会对系统产生多大干扰?
- 如果放任 AI 用方法论概括一切，会产生什么后果?

不能。

原因不是"我们暂时还不够聪明"，而是三类更深层的限制同时存在：

01 对象层限制

现实包含开放系统、路径依赖、涌现与反身性，无法被单一静态规则穷尽。

02 认知层限制

观察者只能获取有限信息，且观察行为本身会改变系统。

03 形式层限制

任何足够强的形式系统，都存在不可完备、不可判定的部分。（哥德尔·图灵·Kolmogorov）

方法论能高度压缩现实、提高预测效率，但无法做到"完全覆盖而无损"。

“

方法论是必要的，

但它永远只是对现实的有损压缩。

越把压缩结果当成全部现实，系统性误差越大。

”

延伸思考

生成与验证的不对称性

P vs NP 的直觉 — 被大语言模型实证强化 (Cobbe et al., 2021)

生成一个正确的解

难

验证一个解是否正确

容易 相对而言

你不需要在实现能力上超越机器——
你需要在评判能力上超越它。

对 AI 时代的启示

AI 的优势在生成端

更快、更多方案、不疲劳、全天候

人的优势在验证端

判断方向对不对、架构好不好、取舍值不值

能力中心的转移

从「如何实现」到「什么叫正确实现」

如果放任 AI 用方法论概括一切……

认知上

· 虚假确定性 & 过度自信

· 误把地图当领土

组织上

· 指标主义 & 模板崇拜

· 异常感知能力退化

社会上

· 解释权集中化

· 复杂性被压平 → 系统脆弱性上升

如何应对

1. 所有方法论都必须标注边界条件
2. 系统里必须保留"未解释剩余项"
3. 不仅看平均效果，还要看尾部风险
4. 允许现实修正框架，
而不是强迫现实服从框架

地球与自然

长期受物理定律约束

高度约束

规律可预测，边界固定

人类文明

存在趋势，但充满分岔

中度开放

路径依赖 + 涌现 + 偶发事件

个人

未来高度开放

极度开放

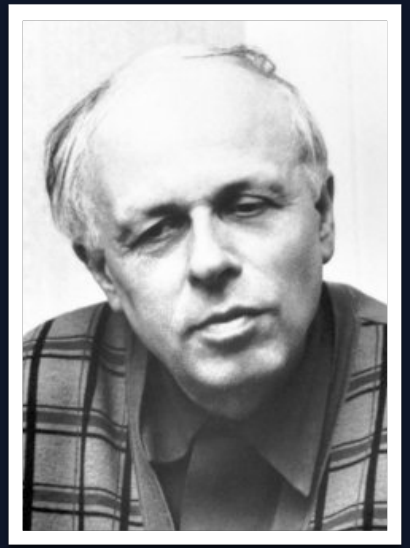
意志 + 选择 + 创造

宇宙决定边界，社会形成趋势，个人创造路径。

数千年前，人类各部落，在苦难里挣扎。活下去，不只靠一根棍棒。还要会用理性想，要想得清楚。要把部落积累的知识与经验，好好守住，传下去。要懂得与他族结盟，彼此扶持。

今天，整个人类，又面临同样的考验。宇宙无边，文明必多。其中，有比我们更聪慧、更"成功"的。我信一种宇宙学的推想：宇宙的演化，在根本特征上，无限次地重演。照此说来，别的文明——包括那些更"成功"的——也该在"宇宙之书"前前后后无数的页面上，一次又一次地出现。

然而，这不该令我们心灰。我们在这个世界上，自有我们神圣的事要做。我们是黑暗里的微光——从物质的沉睡与虚无中，一闪而现。这一现，短暂。但我们来了。我们必须听从理性的吩咐。去创造一种生活，配得上我们自己，配得上那些我们还看不真切、却隐隐感知到的——目标。



— 安德烈·萨哈罗夫 / *Andrei Sakharov, Nobel Prize lecture, 1975*

Q & A

谢谢大家 · Thank you!

有什么问题，欢迎一起讨论 · Questions & discussion welcome

可验证
Verifiable

可复现
Reproducible

可上线
Shippable

参考 / Resources

Skill 环境	github.com/bryanzk/MyCodexEnv
Superpowers	github.com/obra/superpowers
AI 中文推	x.com/dotey
HBR 研究	hbr.org/2026/02/ai-intensifies